railora.

COMPUTATIONAL THINKING
AND PROGRAMMING

SMART TRAVELLING SHORTEST PATH BOOKING PLATFOEM

Methods. Implementation. Evidence.

1. **Decomposition**
2. **Abstraction**
3. **Pattern Recognition**
4. **Algorithm Design**

SHAIK GOUSE BASHA • 2601150020

AI&DS • K L UNIVERSITY

DR. G. Syam Prasad • 2026



Python-authored UI / PostgreSQL CTPDB

Swiss scenic inspiration • Pexels 32548690 • Academic simulation

Decomposition: one problem, eight parts

METHOD Break a complex booking request into focused responsibilities and explicit contracts.

REAL-WORLD QUESTION **Who owns the final seat when several travelers click “Book”?**

1 Identity
Account, session and role

AuthService

2 Discovery
Route, date and class

RailService.search

3 Pricing
Exact fare and service fee

FarePolicy / fare_quote

4 Allocation
Free, unique group seats

allocate_seats

5 Persistence
Booking + seats + event

Database.transaction

6 Lifecycle
Confirm / cancel / expire

RailService

7 Observation
Admin register and charts

AuditEvent / AdminUI

8 Validation
Properties, races and E2E

tests/ + reports/

IN RAILORA A two-passenger request crosses identity, search, fare, seats and atomic persistence—not one giant UI handler.

Abstraction: keep only what matters

METHOD Model essential booking facts; deliberately leave unrelated detail outside the model.

KEEP IN THE MODEL

Train number and endpoints
Journey date and travel class
Passengers, price and status
Seat identity and owner
Deadline and request key

FILTER

LESS DETAIL

CLEARER CONTRACT

OMIT / SIMPLIFY

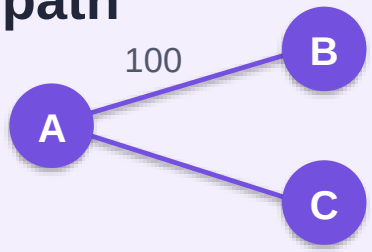
Engine and maintenance details
Live national inventory
Operating calendars
Transfer feasibility
Real payment settlement

IN RAILORA Train + date + class define inventory; station edges support teaching routes, not connection-valid tickets.

Pattern recognition: reusable solutions

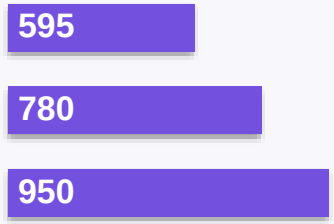
METHOD Recognise familiar structures before choosing code: paths, queues, ordering and shared resources.

Weighted path



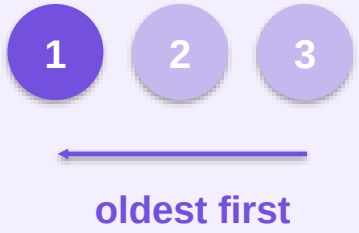
Station → service → station

Stable ordering



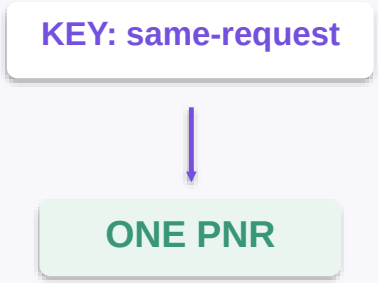
Fare / duration / departure

FIFO fairness



Oldest group stays first

Retry identity



Same key → same PNR

Scarce resource



One seat → one owner

IN RAILORA A weighted graph finds paths; FIFO preserves fairness; idempotent commands make retries safe.

Algorithm design: match method to need

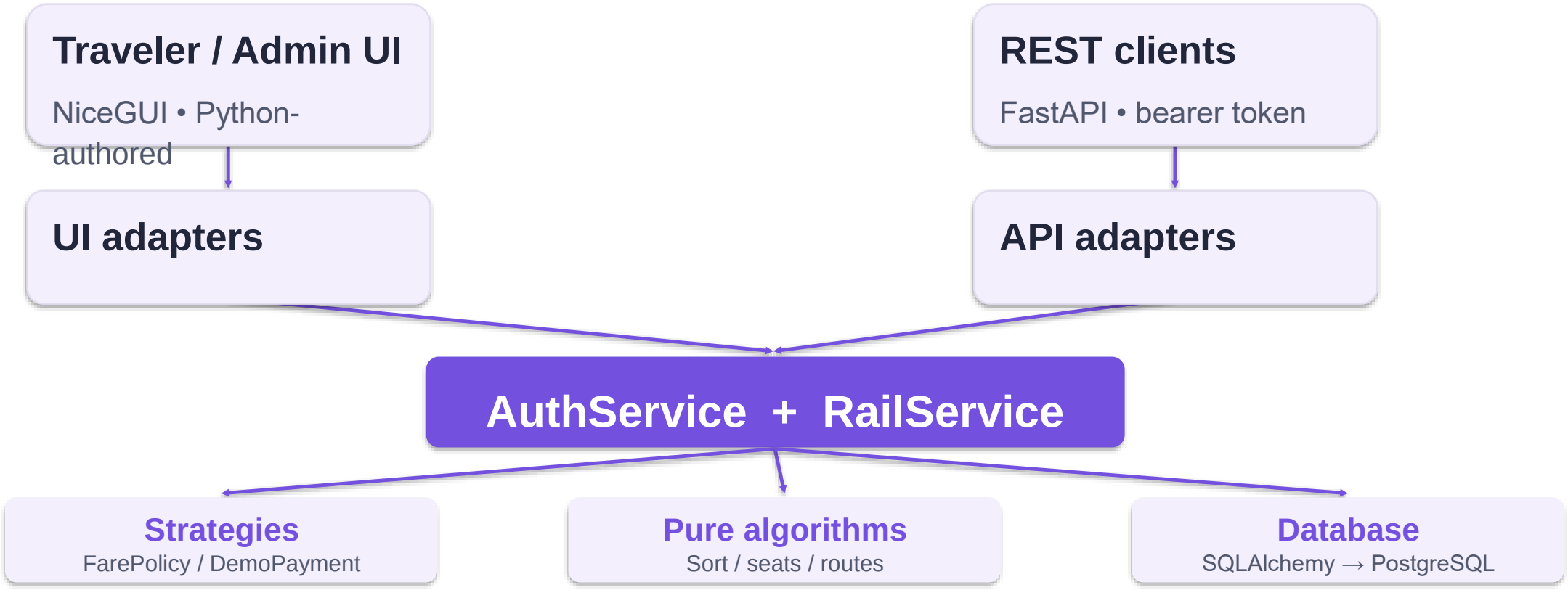
METHOD Select by objective, constraints and worst-case cost—not by using every method everywhere.

METHOD	OBJECTIVE	FUNCTION	WHERE
Divide & conquer	Stable train ranking	`merge_sort`	Search
Greedy	Cheapest non-negative route	`greedy_route`	Planner
Dynamic programming	Cheapest route with ≤H edges	`dynamic_route`	Lab
Backtracking	Find contiguous group seats	`allocate_seats`	Booking
Branch & bound	Fastest simple path within budget	`branch_bound_route`	Lab

IN RAILORA Merge sort and seat allocation are in search/booking; graph optimisers live in the educational planner.

Architecture: two adapters, one rulebook

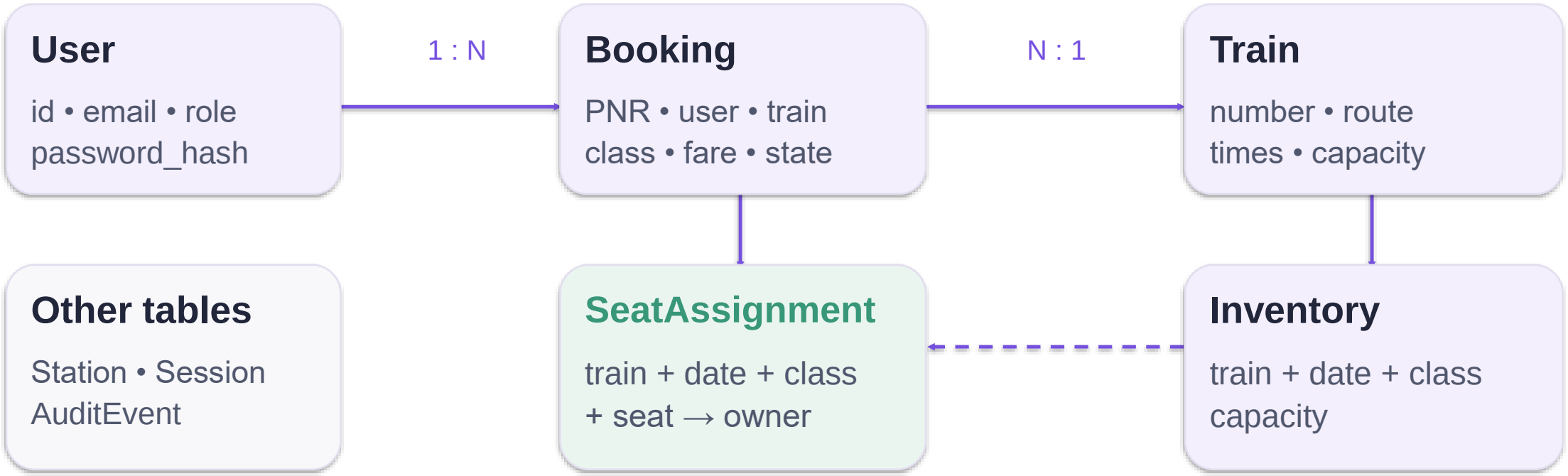
METHOD Use layered modules and shared services to avoid duplicated business rules.



IN RAILORA NiceGUI UI and FastAPI call the same AuthService / RailService; PostgreSQL commits durable state.

Database: model ownership, not counters

METHOD Use related entities, composite identities and database constraints to preserve invariants.



Solid: selected foreign keys • Dashed: logical inventory grouping, not a composite FK

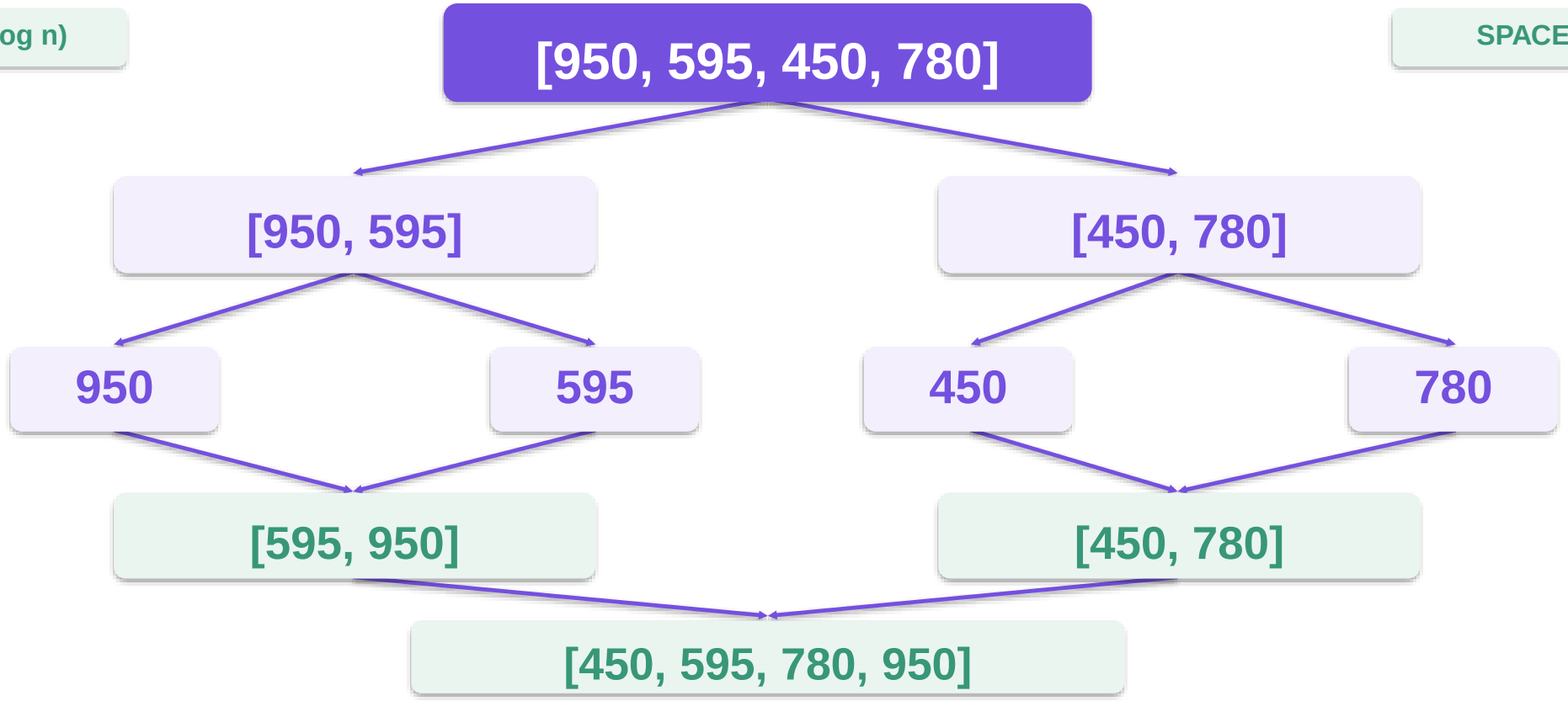
IN RAILORA SeatAssignment PK = (train, date, class, seat); no second committed row can own that same seat identity.

Divide & conquer: split, sort, merge

METHOD Recursively split a list, then stably merge sorted halves.

TIME O(n log n)

SPACE O(n)

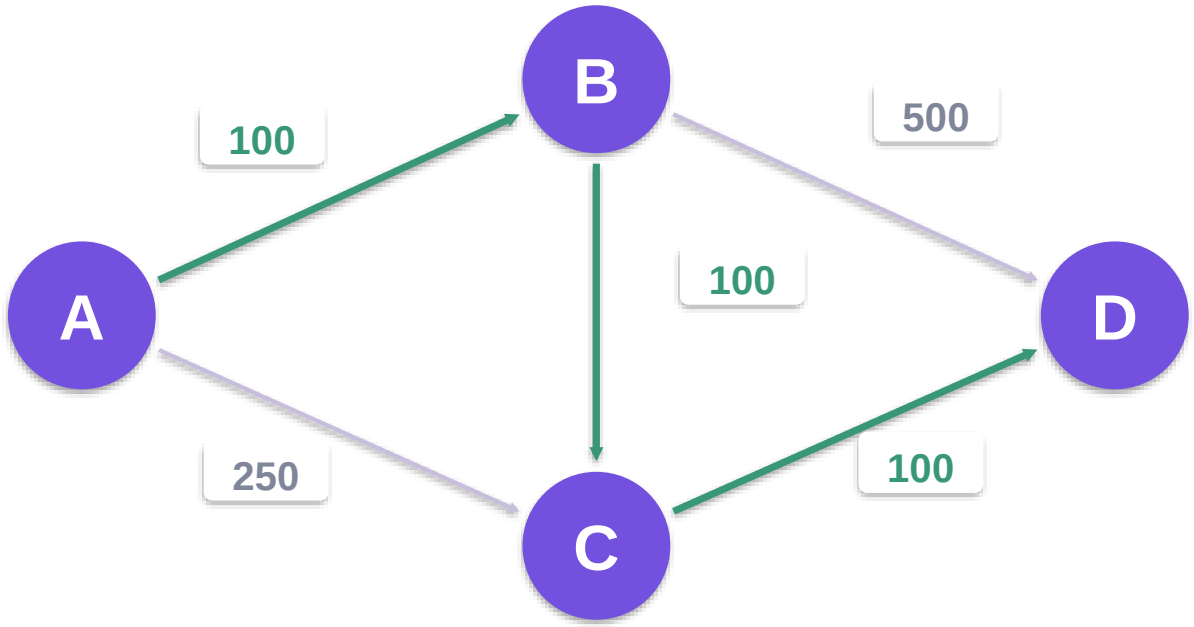


IN RAILORA merge_sort ranks train results by fare, duration or departure; equal keys keep their original order.

Greedy: settle the cheapest frontier

METHOD Dijkstra expands the lowest cumulative base-fare candidate; all edge costs must be non-negative.

UNIT-TEST GRAPH / BASE FARE (INR)



Cheapest route

A → B → C → D

300
total base fare

170
travel minutes

Not “cheapest next edge”

Compare cumulative costs; reject negative weights.

IN RAILORA greedy_route uses heapq + distances + predecessors in the Route Explorer; it does not book connections.

Dynamic programming: bound the legs

METHOD Reuse the previous layer to minimise cost with at most H edges.

H = 0
A : 0

→

H = 1
B : 100
C : 250

→

H = 2
C : 200
D : 350

→

H = 3
D : 300

At most 2 edges

A → C → D

fare 350

At most 3 edges

A → B → C → D

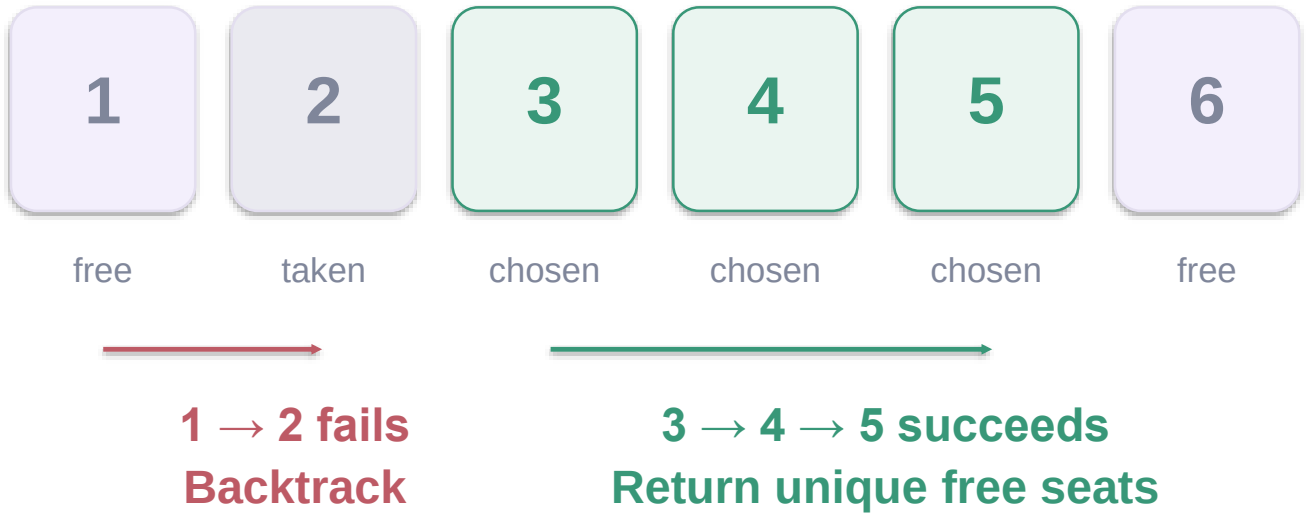
fare 300

IN RAILORA dynamic_route uses previous/current dictionaries; the lab compares ≤4 edges, while tests vary H.

Backtracking: find seats together

METHOD Explore row candidates, prune infeasible prefixes, then use a deterministic fallback.

Example: available {1, 3, 4, 5, 6}; group size = 3



Bound the hard search

Fixed row width = 6

If no row group fits

Choose earliest free seats

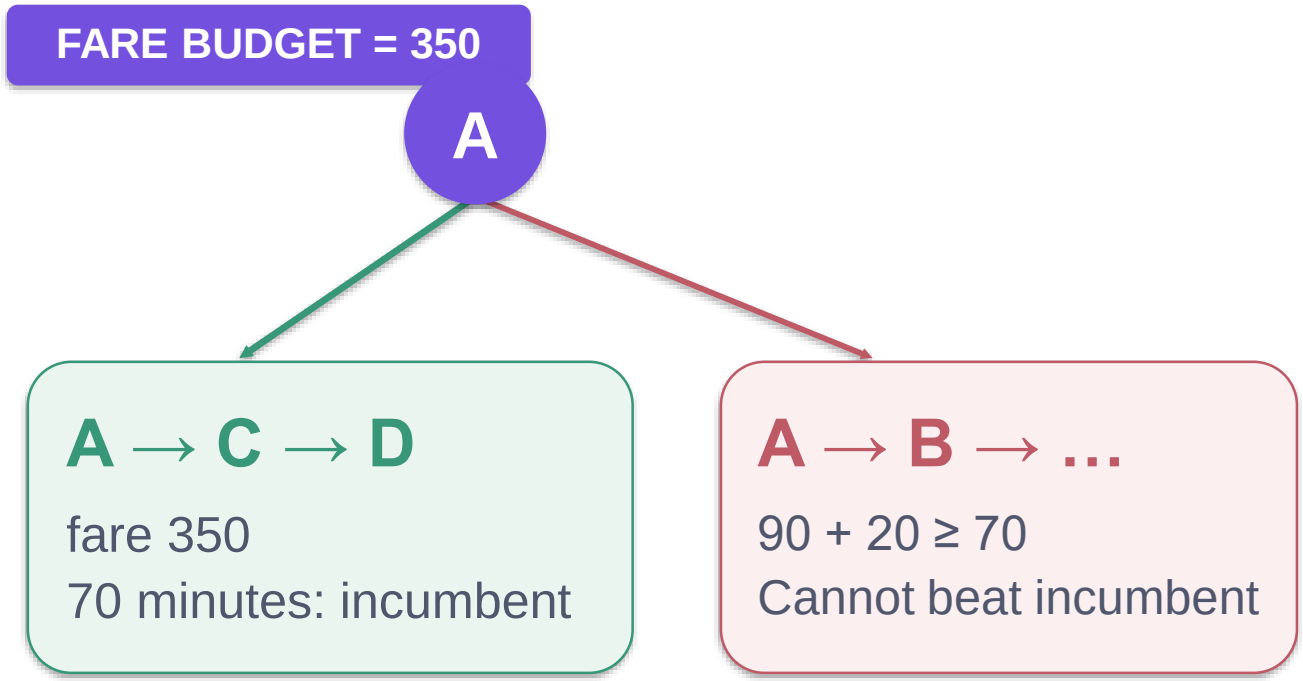
Availability is the invariant

Together is a preference—not a promise.

IN RAILORA allocate_seats searches six-seat rows for a 1–6 person group; together/window remain preferences.

Branch & bound: fastest within a budget

METHOD Search simple paths; reject over-budget branches and prune by an admissible remaining-time bound.



Lower bound helps typical cases; worst-case growth still matters.

Lower the budget to 300

A → B → C → D
fare 300 • 170 minutes

Reject cycles

Only simple paths are explored

Safety cap ≠ optimum

20,000 states → error if exceeded.

IN RAILORA branch_bound_route minimises minutes under a fare budget; a 20,000-state cap raises an explicit error.

Complexity: analyse the actual code

METHOD Distinguish worst-case bounds, expected hashing and amortized resizing from measured latency.

ROUTINE	TIME / QUALIFICATION	AUXILIARY SPACE
Merge sort	n log n	n peak + log n stack
Dijkstra	(V+E) log(V+E); lazy heap	V+E
Bounded DP	H(V+E) + up to H^2E path-copy work	HV stored paths
Seat backtracking	n log n + n·64; fixed 6-seat rows	n; depth ≤6
Branch & bound	Exponential/factorial; VE bound prep	V^2 copied prefixes + E

EXPECTED Set/dict lookup: O(1) expected, not guaranteed.

AMORTIZED List append: O(1) averaged across resize costs.

IN RAILORA Path-tuple copies count: DP stores full paths, and branch recursion retains several copied prefixes.

Data structures: choose for the operation

METHOD Use ordered sequences, membership sets, priority heaps, FIFO queues and immutable values purposefully.

List

Ordered passengers / results

`[p1, p2, …]`

Set + dict

Free-seat membership / distances

`{1, 3, 4}  best[A]=0`

Heap

Lowest-cost route frontier

`100 → 200 → 300`

Deque

Oldest waitlist group first

`head → group → group`

Frozen values

Edges / paths as value objects

`Edge • RouteResult`

DB key + index

Cross-process seat identity

`train + date + class + seat`

IN RAILORA Free seats use sets; Dijkstra uses heapq; waitlist traversal uses deque; PostgreSQL keys enforce ownership.

Pythonic programming: concise, not cryptic

METHOD Use generators, comprehensions, decorators and context managers without hiding side effects.

ITERATOR / GENERATOR

```
labels = seat_labels([1, 37])
next(labels)  # C1-01
next(labels)  # C2-01
```

COMPREHENSION

```
free = [seat for seat in pool
        if seat not in taken]
```

DECORATOR

```
@timed
def quote(base, cls, people):
    return fare_quote(
        base, cls, people)
```

CONTEXT MANAGER

```
with db.transaction() as session:
    session.add(booking)
# success: commit
# exception: rollback
```

IN RAILORA seat_labels is lazy; timed preserves the callable; Database.transaction commits or rolls back reliably.

OOP: compose replaceable responsibilities

METHOD Define abstract contracts, inherit concrete policies and inject collaborators by composition.



IN RAILORA RailService composes Database, FarePolicy and PaymentGateway; UI/API consume its shared behavior.

Types and generics: make contracts visible

METHOD Express reusable types, structural protocols and immutable values; validate runtime inputs separately.

REUSABLE GENERIC REPOSITORY

```
class Repository(Generic[T]):
    def add(self, entity: T) -> T: ...
    def all(self) -> list[T]: ...
```

SIGNATURE-PRESERVING DECORATOR

```
def timed(
    fn: Callable[P, R]
) -> Callable[P, R]: ...
# P = ParamSpec; R = TypeVar
```

Static checking

Mypy: 7 core modules

Runtime validation

Pydantic + service invariants

Immutable values

Frozen Edge / RouteResult

Typed boundaries: UI / API → schemas → services → models / pure algorithms

IN RAILORA Mypy checks seven core modules; Pydantic validates passenger and train commands at runtime.

Functional design: separate values from effects

METHOD Use pure calculations and higher-order functions; isolate database and payment effects.

Input values

base = 595
class = CC, people = 2

→ **fare_quote()** →

Output value

subtotal 1,190 + fee 40
TOTAL 1,230

HIGHER-ORDER KEY FUNCTION

```
ranked = merge_sort(
    trains, key=lambda t: t["fare"])
```

Effects stay in services

Persist booking + seats + audit.
Simulated payment is not a pure calculation.

IN RAILORA fare_quote, sort, allocation and route functions are testable values; services own state transitions.

Concurrency: select by workload

METHOD Threads overlap blocking I/O; async coordinates tasks; processes run pure Python CPU work.

I/O

THREADS

Blocking DB / service I/O

`run.io_bound(...)`

AWAIT

ASYNC

Coordinate independent reads

`asyncio.gather(...)`

CPU

PROCESSES

Pure graph / CPU computation

`run.cpu_bound(...)`

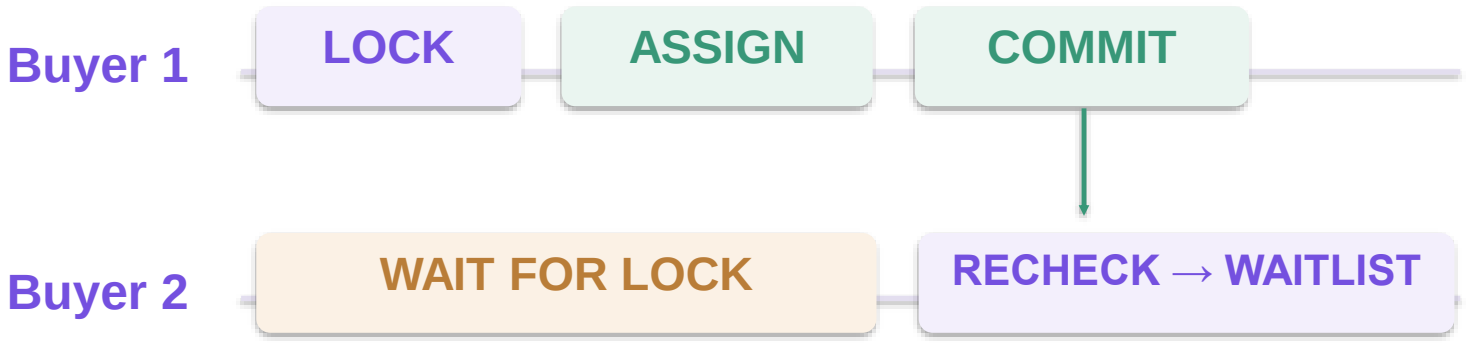
GIL insight
Pure Python CPU loops → processes. Blocking I/O → threads. Async ≠ a lock on scarce seats.

IN RAILORA DB calls use run.io_bound; dashboard reads use asyncio.gather; algorithm lab uses run.cpu_bound.

Atomic booking: protect the final seat

METHOD Combine row locks, command identity and unique constraints inside one transaction.

ILLUSTRATIVE TIMELINE / ONE SEAT REMAINS



Idempotency

same key + same payload
→ same PNR

Final DB guard

one composite seat key
→ one assignment

Booking + SeatAssignment + AuditEvent

Exception → rollback all three

IN RAILORA Lock order is Train → Inventory → Booking; booking, seats and audit commit or roll back together.

Lifecycle: holds, expiry and fair promotion

METHOD Model explicit states; release seats safely and serve the oldest feasible queue head first.



FIFO EXAMPLE 2 free seats; the oldest group needs 3

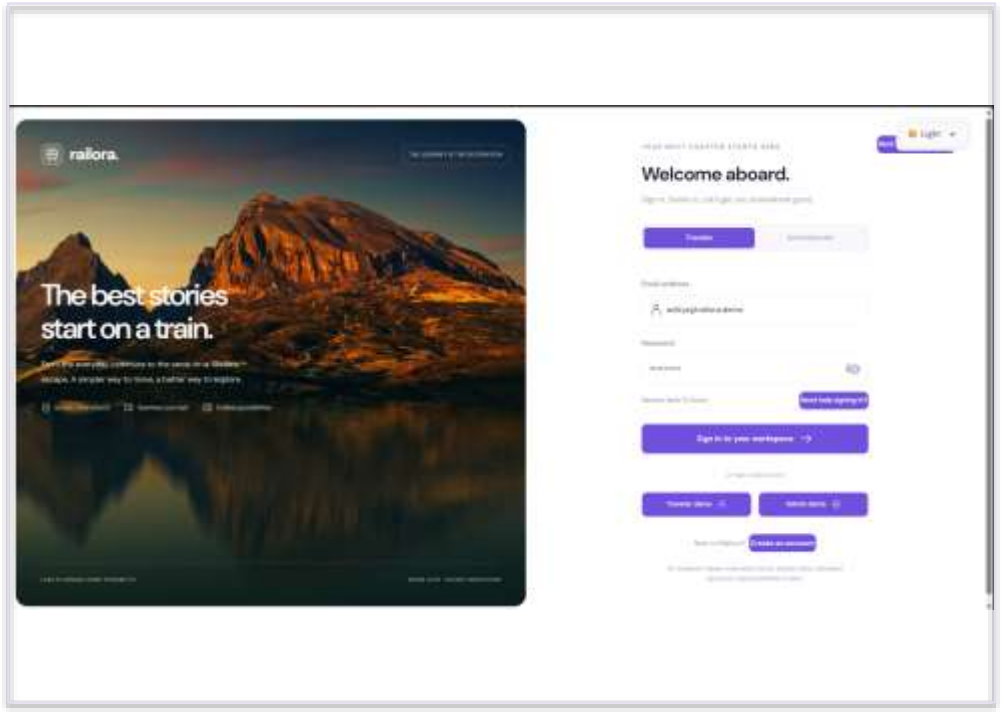


Head blocks younger groups: fairness over throughput.

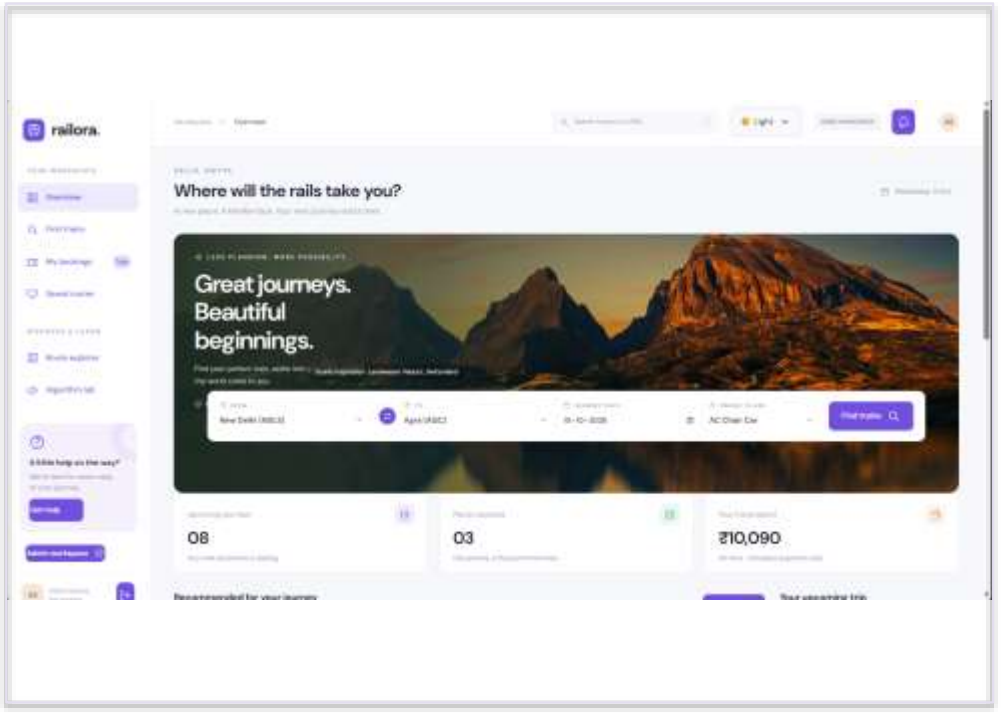
IN RAILORA A promoted waitlist group becomes reserved for 15 minutes—not automatically paid or confirmed.

Traveler UI: make the method observable

METHOD Use clear feedback, responsive layout and shared validated commands at the presentation boundary.



Role-aware sign-in • local real photography

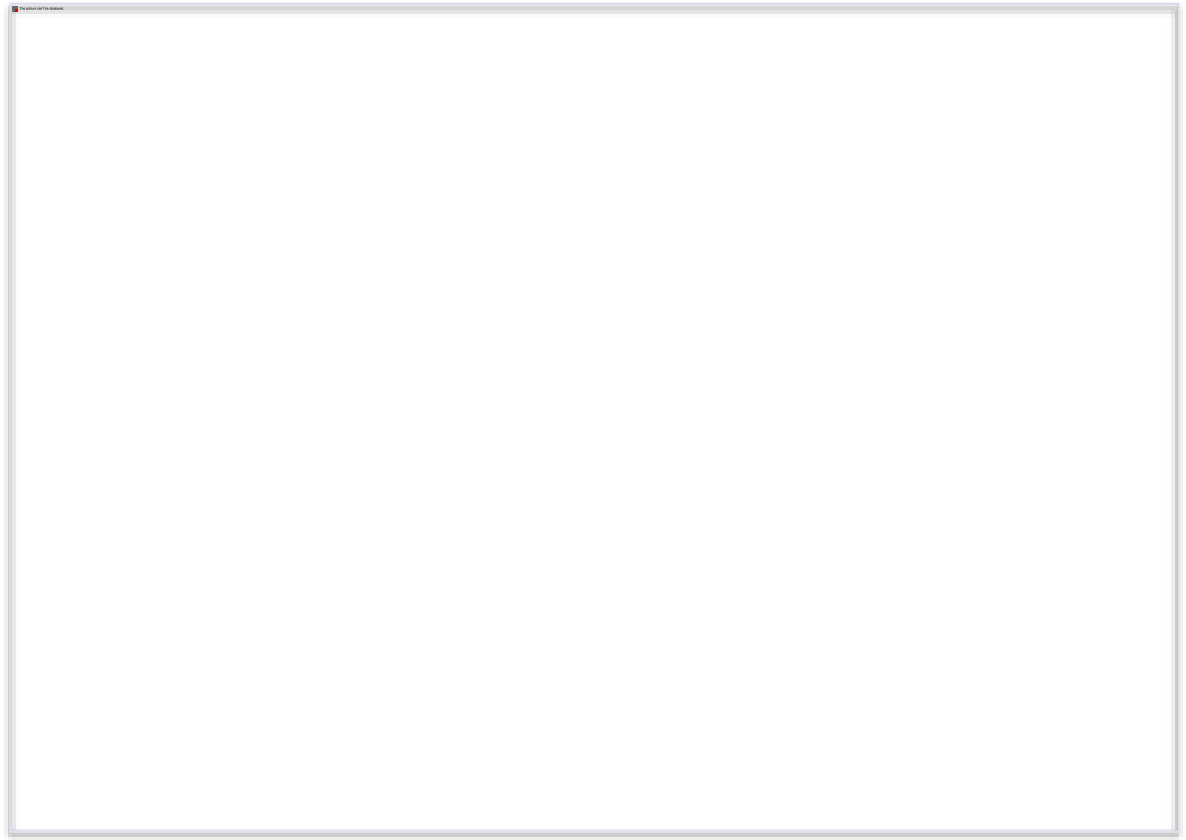


Direct train discovery • passenger booking

IN RAILORA Login → search → passenger modal → PNR/demo ticket; the service rechecks inventory at submit time.

Admin UI: observe committed changes

METHOD Combine role-based access with periodic reads of durable statistics and audit events.



Role checks

UI and services enforce admin

Persistent updates

Booking + audit commit first

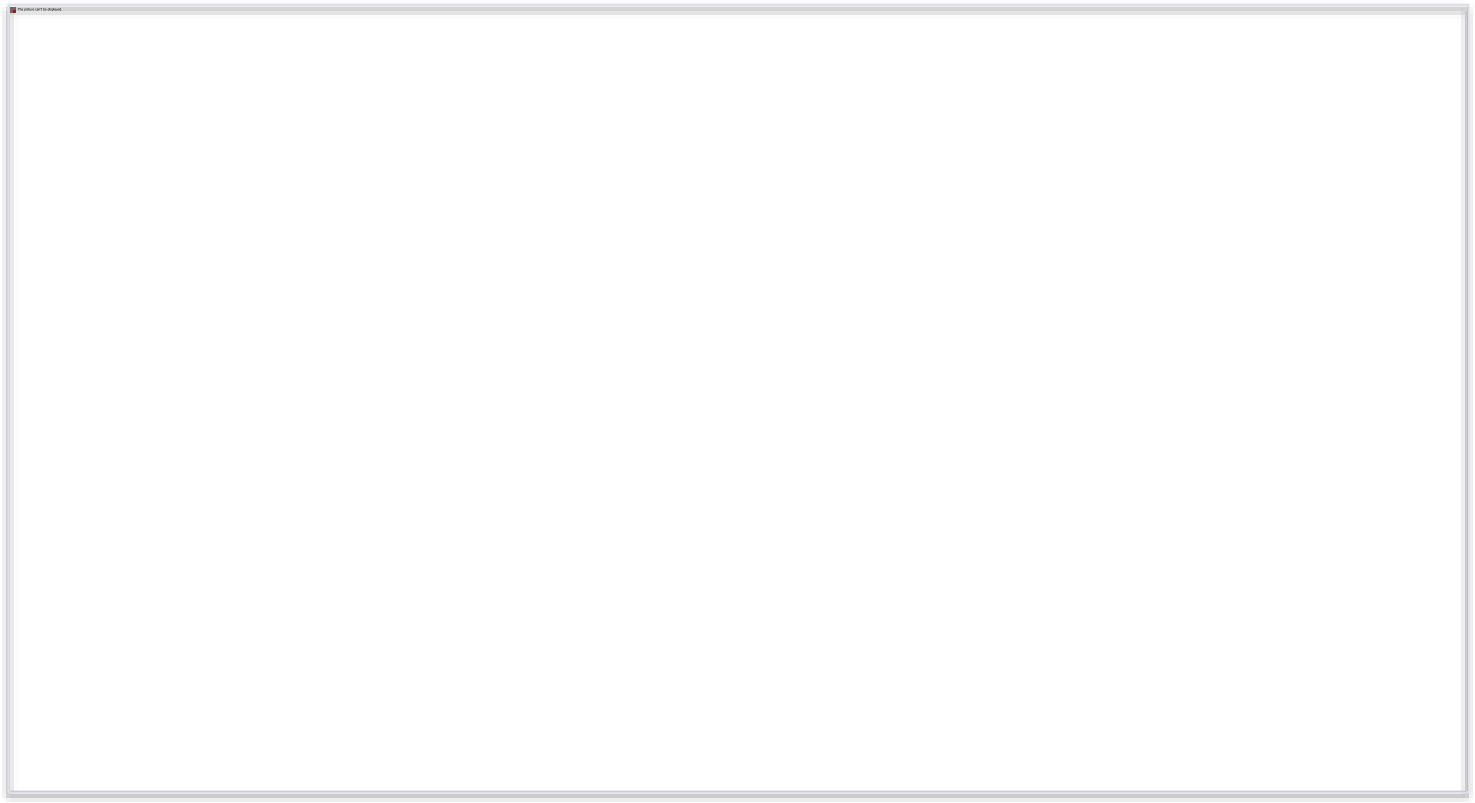
Polling + WebSocket

Refresh about every 3 seconds

IN RAILORA AdminUI polls about every 3 seconds; charts/register reflect synthetic CTPDB data, not browser-only counters.

Algorithm Lab: compare the same network

METHOD Run pure graph computations in a process pool and compare their different objectives and constraints.



GREEDY

Lowest base fare

DYNAMIC PROGRAMMING

Lowest fare, ≤4 edges

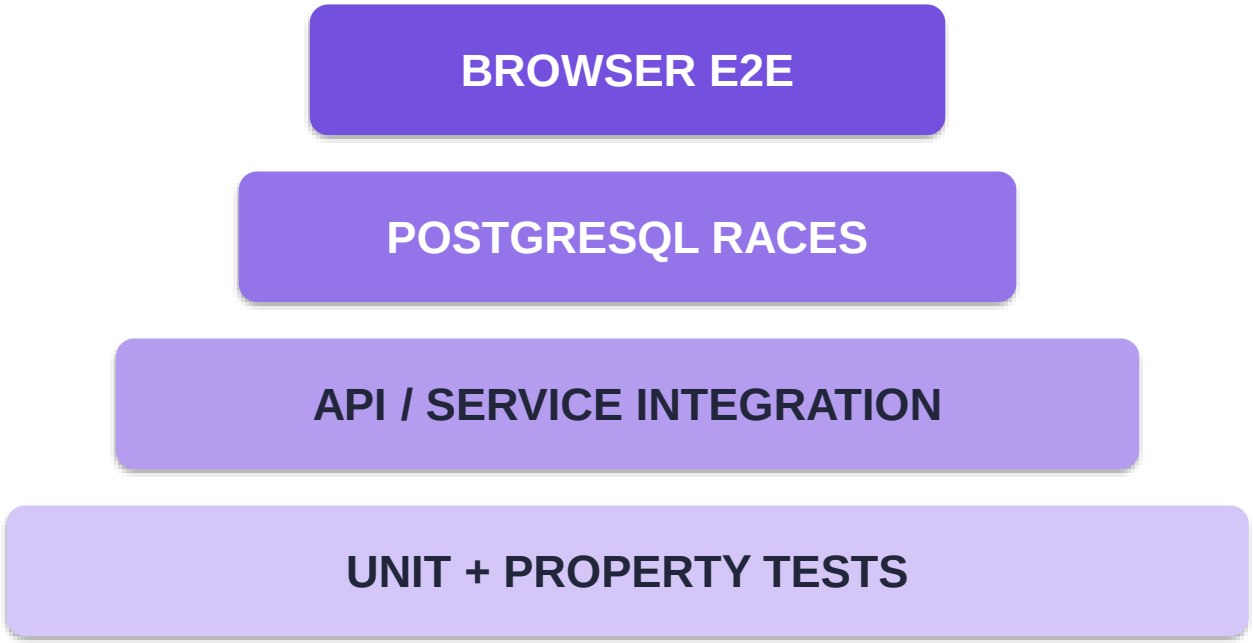
BRANCH & BOUND

Fastest within budget

IN RAILORA Dijkstra / DP return base-fare cost; branch-and-bound returns minutes. The seat sandbox reserves nothing.

Testing: examples plus independent oracles

METHOD Layer unit, service, API, database-race and browser tests; generate boundary inputs with Hypothesis.



Hypothesis generates

Seat lists, group sizes and graphs

Check invariants

unique ∧ free ∧ valid result count

Small graphs → independent brute force
Failing examples → shrink and inspect

IN RAILORA Allocation properties check free/unique seats; route properties compare against independent brute-force results.

Quality evidence: show the scope

METHOD Report executed checks accurately; coverage supports review but is not a correctness proof.

49
PostgreSQL cases passed
4 browser cases skipped here

4
Browser E2E cases passed
Separate browser invocation

98.49%
Core statement coverage
846 / 859 • UI and seed excluded

Ruff lint
PASSED

Formatting
PASSED

Mypy
7 core modules

Compilation
PASSED

Scope matters
SQLite repeats the core cases. Coverage is not full-UI or branch coverage; a deprecation warning remains.

IN RAILORA 49 PostgreSQL cases + 4 browser cases = 53 distinct cases; scoped core statement coverage is 98.49%.

DevOps: configured gates, honest status

METHOD Automate repeatable checks and container configuration; do not confuse configuration with execution.

Commit / PR → Lint + format → Mypy → PG tests / coverage → Container build

PARALLEL JOB: Chromium browser E2E

Application container

Non-root Python / NiceGUI

→

PostgreSQL container

Health check + persistent volume

STATUS: configured + structurally checked; not remotely executed or automatically deployed.

IN RAILORA CI defines quality/PostgreSQL and browser jobs; Dockerfile + Compose are included, but remote/container runs are unverified.

AI-assisted: specify before generating

METHOD Supply narrow context, explicit invariants and falsifiable tests; treat generated code as a draft.

EXAMPLE PROMPT TEMPLATE • NOT A TRANSCRIPT

```
TASK: implement one booking command
CONTEXT: schema + service + relevant tests
MUST PRESERVE:
  unique seat ownership
  atomic booking / seats / event
  retry identity + FIFO + role checks
DELIVER: plan, code, edge cases, tests
```

Narrow context

Contracts, not blind rewrites

Generate a draft

Explain assumptions first

Validate critically

Run independent checks

IN RAILORA Use schemas + lock order + constraints to request a bounded change, then inspect and validate it.

AI review: defects found, not hidden

METHOD Inspect actual behavior, identify counterexamples, fix the cause and rerun checks.

UI state

Committed booking; success dialog invisible

Browser workflow
+ DB/API inspection

↓

Root dialog outside closed parent

CSS collision

Admin rows collapse into table-like layout

Visual layout
+ computed style

↓

Rename table-row to data-row

PostgreSQL

Fixture IDs do not advance sequences

First PG run:
4 failing cases

↓

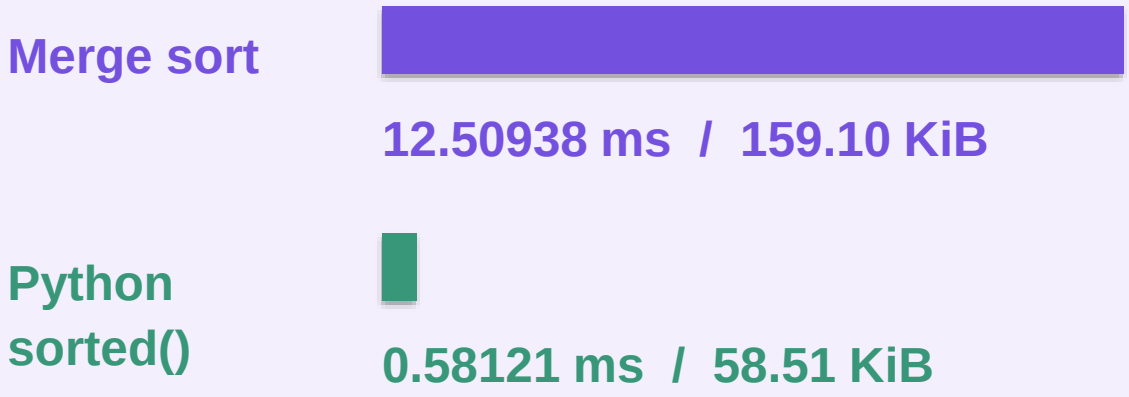
Use DB-generated IDs; rerun succeeds

IN RAILORA Browser inspection and PostgreSQL tests caught UI-state, CSS and fixture defects in generated drafts.

Performance: measure, then improve

METHOD Benchmark reproducibly and disclose input size, repeats, memory scope and excluded costs.

Same 5,000 items
Median per call over 5 repeats



Warm local calls; no browser / network / worker transport.

3.54711 ms
DB search: 15 services

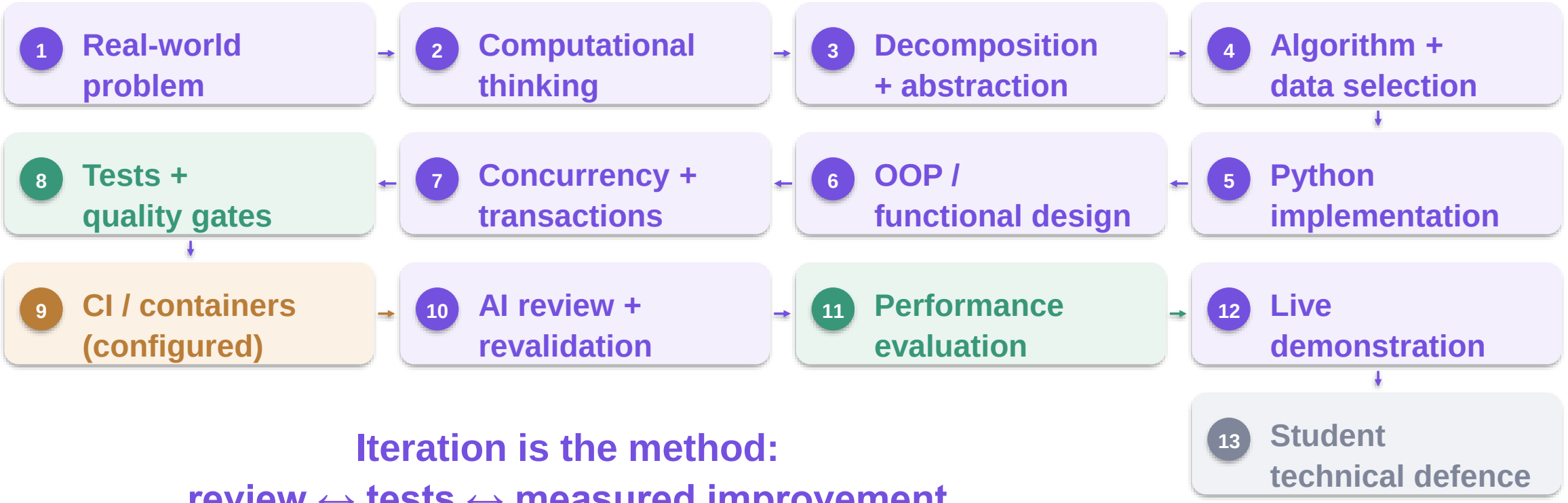
89.62213 ms
Admin: 46 bookings

Next: SQL pagination, joined/batched loading, indexed aggregates.

IN RAILORA Use native sorted() as a reference; prioritise SQL pagination/batched loading for the current admin bottleneck.

The complete Computational Thinking workflow

METHOD Trace decisions from a real-world invariant to validated code, measured behavior and defence.

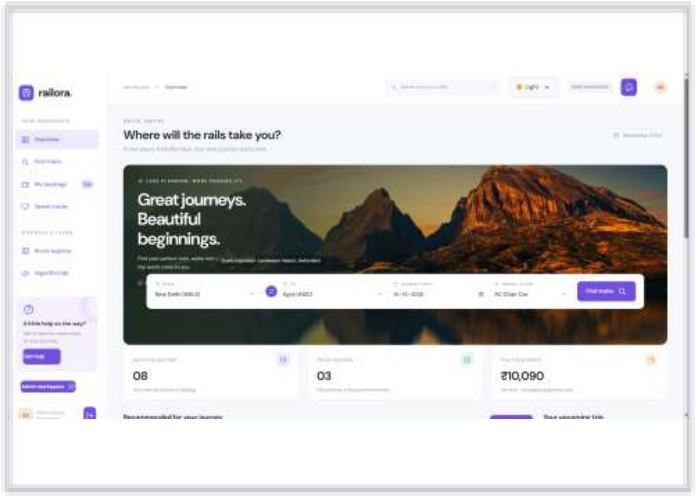


Iteration is the method:
review ↔ tests ↔ measured improvement

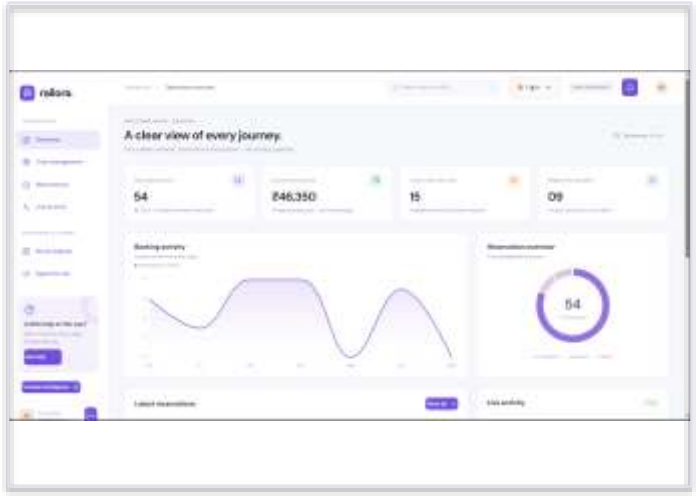
IN RAILORA AI review and testing repeat throughout; CI/containerisation is configured, while student defence remains required.

Live demo: one journey, two sessions

METHOD Demonstrate observable state changes with separate traveler and administrator profiles.



TRAVELER: BOOK / CANCEL



ADMIN: OBSERVE COMMITTED DATA

Search → Book 2 → PNR / PDF → Admin update → Cancel → Run lab

IN RAILORA Search → book two passengers → PNR/demo PDF → admin update → cancel → algorithm lab.

Limitations: turn honesty into a roadmap

METHOD Evaluate abstraction gaps and bottlenecks before proposing targeted, tested improvements.

ACADEMIC NOW

Direct sample routes
Simulated fares/payment
One web process

ENGINEERING NEXT

SQL pagination + batching
Migrations + safe seeding
TLS, secrets, recovery

AUTHORISED RAIL

Segment + calendar inventory
Feasible transfers
Payment reconciliation

Measure → change one boundary → add counterexamples → revalidate

IN RAILORA Railora is an academic simulation—not official booking, real payment settlement or a production deployment.

Technical defence: explain the decision

METHOD Answer with invariants, algorithm preconditions, source contracts and executed evidence.

Why not trust availability?

Snapshot ≠ ownership.
Lock + unique key at commit.

Why can DP cost differ?

The allowed edge count changes
which paths are feasible.

Why bounded seat search?

Six-seat rows; k≤6.
Preferences can fall back.

Why processes for the lab?

Pure Python CPU work and GIL.
Measure process overhead.

Does 98.49% prove safety?

No: scoped statement coverage.
Review failures and properties.

What proves mastery?

A new regression + small change
+ explanation of its invariants.

IN RAILORA A student should defend these choices and make one meaningful, tested extension—not merely show the UI.

Every course outcome has inspectable evidence

METHOD Link each claim to code, test artefacts or a demonstration; distinguish completed from proposed work.

01 Computational thinking

Decomposition → five patterns
Time / space complexity
Expected and amortized

algorithms.py / ALGORITHMS.md

02 Python and data

Generators + comprehensions
Decorators + contexts
Measure Python performance

algorithms.py / db.py / benchmark.py

03 Software design

ABC / inheritance / Protocol
Composition + design patterns
Type hints + generics

services.py / schemas.py / db.py

04 Functional / concurrency

Pure + higher-order functions
Threads / processes / async
CPU vs I/O and the GIL

algorithms.py / UI modules

05 Quality / DevOps

Pytest + Hypothesis + E2E
Ruff + Mypy + coverage
CI / containers: configured

tests/ / reports/ / ci.yml

06 Responsible AI

Specification-first prompts
Scoped context + generation
Review → tests → improve

AI_REVIEW.md / regression evidence

IN RAILORA Use the source ZIP + project report + this deck together; fill the cover details and rehearse the defence.